

DOCUMENTO TÉCNICO DE IMPLEMENTACIÓN

Equipo 27

Julian David Miranda Leguizamón

Sebastian Bedoya Florez

Edgar Andres Romero Otalora

Jhon Alexander Tenjo Romero

Universidad de la Sabana

Maestría en Arquitectura de Software

Miguel Alfonso Varela Fonseca

Bogotá D.C.

Índice

Resumen Ejecutivo	3
Síntesis de implementación realizada	3
Principales optimizaciones aplicadas.	3
PostgreSQL	3
MongoDB	4
Resultados cuantitativos destacados	4
PostgreSQL	4
MongoDB	5
Implementación PostgreSQL	6
Estrategia de indexación	6
Particionamiento	7
Implementación MongoDB	8
Products	8
Esquemas de documentos	8
Índices implementados	9
Aggregation pipelines	10
Order_reviews	11
Esquemas de documentos	11
Índices implementados	12
Aggregation pipelines	13
Geolocation	14
Esquemas de documentos	14
Índices implementados	15
Aggregation pipelines	16
Replica Sets	17
Sharding	17
Evidencias cuantitativas de mejoras de rendimiento	19
PostgreSQL: Tablas comparativas y gráficas de mejora	19
MongoDB: Métricas de executionTimeMillis y efficiency ratios	20
Products	20
Order Reviews	21
Geolocation	21
Interpretación de resultados y análisis de impacto	21
Products	21
Order Reviews	22
Geolocation	22
Sincronización entre sistemas	24
Flujos de Sincronización	24
Sincronización Transaccional a Consulta (Ida: PostgreSQL → MongoDB):	24

Sincronización de Interacción a Analítica (Vuelta: MongoDB → PostgreSQL):	24
Estrategia de consistencia	25
Lecciones aprendidas	26

Resumen Ejecutivo

Síntesis de implementación realizada

Se estableció una metodología de optimización sobre dos motores de base de datos complementarios: PostgreSQL, como sistema relacional transaccional, y MongoDB, como sistema documental orientado a consultas analíticas, utilizando el dataset del proyecto Ecommify (100.000 registros).

Para PostgreSQL (validado en entorno local, proyectado a Supabase y Colab), la metodología se dividió en dos fases:

- Línea base: Se trabajó sin índices adicionales, utilizando únicamente las restricciones definidas por las llaves primarias y foráneas, con el fin de identificar con mayor precisión los problemas de rendimiento reales de la base de datos.
- Análisis de consultas: Mediante el uso de EXPLAIN ANALYZE se identificaron y documentaron los cuellos de botella del planificador.

Para MongoDB, la metodología se estructuró sobre tres colecciones principales: Products, Order Reviews y Geolocation. En cada una se aplicó un proceso de rediseño de esquema, definición de índices optimizados y construcción de aggregation pipelines eficientes. Adicionalmente, se implementó una arquitectura de alta disponibilidad mediante Replica Sets con grupos de tres nodos por segmento de datos, y se diseñó una estrategia de Sharding para soportar el crecimiento futuro de la plataforma.

Principales optimizaciones aplicadas.

PostgreSQL

- Indexación de relaciones: Se implementan índices b-tree sobre las llaves foráneas de las base de datos, eliminando la necesidad de que el motor realizará costosos cruces globales en memoria (Hash joins).

- Indexación invertida para búsquedas difusas: Se habilita la extensión pg_trgm y se crean índices GIN para optimizar las consultas con patrones LIKE.
- Mitigación de desbordamiento en disco: Se crea un índice b-tree en columnas de control operativo, diseñado específicamente para acelerar reportes de alta selectividad eliminando archivos temporales en el almacenamiento secundario.

MongoDB

- Índices compuestos tipo covering index: En la colección Products se implementó un índice compuesto sobre physical_specs.weight_g, product_category_name y performance_metrics.average_review_score, cubriendo la totalidad de las etapas de lectura del pipeline sin accesos adicionales a disco.
- Índice simple sobre campo de filtro de alta selectividad: En Order Reviews se implementó un índice sobre review_score que permite ejecutar un index range scan directo, accediendo únicamente a los documentos con calificación mayor o igual a 4.
- Índice compuesto estado-ciudad en Geolocation: Se implementó un índice sobre state y city que, combinado con la deduplicación de la colección, permitió reducir de 2.000.326 a 6.349 los documentos examinados por consulta.
-

Resultados cuantitativos destacados

PostgreSQL

- Aceleración en búsquedas de texto: La implementación de un índice GIN con la extensión pg_trgm mejoró el rendimiento de las búsquedas parciales realizadas sobre las ciudades. Gracias a esta optimización los tiempos de respuesta mejoraron entre un 50% y 98%.
- Reducción del uso de memoria caché: En la consulta relacionada con el historial del cliente, la creación de índices sobre las llaves foráneas permitió disminuir la cantidad

de bloques leídos desde la caché, permitiendo evidenciar mejoras entre el 20% y el 70% en tiempos de respuesta.

- Mejora en consultas de agregación: Las pruebas realizadas sobre la columna `order_status` demostraron cómo PostgreSQL toma decisiones basadas en costos. Cuando la consulta recuperaba una gran parte de la tabla, con el estado `delivered`, el optimizador optó por un escaneo secuencial, ya que utilizar un índice habría generado más lecturas aleatorias y un mayor costo. Por otro lado, cuando se consultó un valor poco frecuente como `canceled`, el motor utilizó inmediatamente el índice disponible, mejorando el rendimiento de forma considerable hasta en un 60%.

MongoDB

- Products: Los documentos examinados se redujeron en un 58,5% (de 65.902 a 27.356) y el tiempo de ejecución disminuyó de 183 ms a 156 ms (↓ 14,8%), con un Index Efficiency Ratio de 1.00, confirmando que el índice operó sin falsos positivos.
- Order Reviews: Los documentos examinados se redujeron en un 61,7% (de 198.448 a 75.917) y el tiempo de ejecución disminuyó de 372 ms a 113 ms (↓ 69,6%), combinando la limpieza del esquema con el acceso indexado directo.
- Geolocation: La mejora más significativa del proyecto: los documentos examinados se redujeron en un 99,7% (de 2.000.326 a 6.349) y el tiempo de ejecución pasó de 1.783 ms a 11 ms (↓ 99,4%), equivalente a un factor de aceleración de 162 veces, resultado de la deduplicación del esquema combinada con el índice compuesto.

Implementación PostgreSQL

Estrategia de indexación

A continuación, se detalla el script DDL de optimización ejecutado:

```
– Habilitación de extensión para optimización de texto
CREATE EXTENSION IF NOT EXISTS pg_trgm;

– 1. Índices GIN para búsquedas difusas
CREATE INDEX categories_name_trgm_idx ON public.categories USING gin
(category_name gin_trgm_ops);
CREATE INDEX zip_codes_city_trgm_idx ON public.zip_codes USING gin (city
gin_trgm_ops);

– 2. Índices b-tree en llaves foráneas
CREATE INDEX customers_customer_zip_code_idx ON public.customers USING
btree (customer_zip_code);
CREATE INDEX orders_customer_id_idx ON public.orders USING btree
(customer_id);
CREATE INDEX sellers_seller_zip_code_idx ON public.sellers USING btree
(seller_zip_code);
CREATE INDEX order_items_order_id_idx ON public.order_items USING btree
(order_id);
CREATE INDEX order_items_seller_id_idx ON public.order_items USING btree
(seller_id);
CREATE INDEX order_payments_order_id_idx ON public.order_payments USING
btree (order_id);

– 3. Índices b-tree para control operativo
CREATE INDEX orders_order_status_idx ON public.orders USING btree
(order_status);
CREATE INDEX order_payments_payment_type_idx ON public.order_payments
USING btree (payment_type);
```

- Índices b-tree: Se definió para llaves foráneas de las diferentes tablas y campos para control operativo como el campo order_status y payment_type. Esto permite mejorar el rendimiento de búsquedas por igualdad, operaciones de ordenamiento y consultas por rangos, su eficiencia permite localizar registros de manera rápida.

- Índices GIN: Para optimizar las búsquedas parciales de texto se definieron índices tipo GIN. Con esto se busca optimizar las búsquedas realizadas en tablas como `zip_codes` y `categories` dejando de depender de escaneos completos de la tabla.

Particionamiento

Durante la fase de diseño de Ecommify, se evaluó la posibilidad de aplicar particionamiento de tablas, por ejemplo en `orders` o `order_items`, sin embargo, se determinó que no era necesario debido al volumen actual de datos, el cual puede gestionarse de manera eficiente con los índices actuales. Adicionalmente, el sistema realiza consultas analíticas que requieren acceder al histórico completo de información, por lo que el particionamiento podría incrementar la complejidad sin generar beneficios significativos de rendimiento. Por esta razón, se decidió mantener esta estrategia como una opción de escalabilidad para futuras etapas del proyecto cuando el crecimiento de datos lo justifique.

Implementación MongoDB

Products

Esquemas de documentos

```
{
  $jsonSchema: {
    bsonType: 'object',
    required: [
      '_id',
      'product_category_name'
    ],
    properties: {
      _id: {
        bsonType: 'string',
        description: 'El product_id original de Olist es un hash (String)'
      },
      product_category_name: {
        bsonType: 'string',
        description: 'Debe ser un string y es obligatorio (Llave relacional)'
      },
      physical_specs: {
        bsonType: 'object',
        properties: {
          weight_g: {
            bsonType: 'int'
          },
          length_cm: {
            bsonType: 'int'
          },
          height_cm: {
            bsonType: 'int'
          },
          width_cm: {
            bsonType: 'int'
          }
        }
      },
      multimedia_metadata: {
        bsonType: 'object',
        properties: {
          product_name_length: {
            bsonType: 'int'
          },
          product_description_length: {
            bsonType: 'int'
          },
          photos_qty: {
            bsonType: 'int'
          }
        }
      },
      performance_metrics: {
        bsonType: 'object',
        description: 'Implementación del Computed Pattern',
        properties: {
          average_review_score: {
```

Índices implementados

Se implementó un índice compuesto sobre tres campos del esquema optimizado de `olist optimized.products`:

Segundo campo `product_category_name` (ascendente): Al incluir este campo en el índice, la etapa \$project posterior puede leer su valor directamente desde la estructura del índice sin necesidad de acceder al documento completo en disco, habilitando un acceso de tipo `index-covered` para esa etapa.

Los tres campos en conjunto convierten la estructura en un covering index que cubre la totalidad de las etapas de lectura del pipeline.

Decisión técnica sobre `partialFilterExpression`: Una expresión parcial sobre `average_review_score > 0` fue evaluada durante el diseño pero descartada, debido a que el optimizador de MongoDB rechaza utilizar un índice parcial cuando no puede garantizar que todos los documentos del resultado `$match` estén incluidos en él. Al eliminarla, el índice cubre el pipeline de forma completa y determinista.

Aggregation pipelines

```
pipeline_opt = [  
  {  
    "$match": {  
      "physical_specs.weight_g": {  
        "$gt": 200  
      }  
    }  
  },  
  {  
    "$project": {  
      "product_category_name": 1,  
      "weight": "$physical_specs.weight_g",  
      "avg_review": "$performance_metrics.average_review_score"  
    }  
  },  
  {  
    "$addFields": {  
      "weight_category": {  
        "$cond": [  
          {"$gte": ["$weight", 1000]},  
          "Heavy",  
          "Light"  
        ]  
      }  
    }  
  },  
  {  
    "$group": {  
      "_id": {  
        "category": "$product_category_name",  
        "weight_type": "$weight_category"  
      },  
      "avg_weight": {"$avg": "$weight"},  
      "total_products": {"$sum": 1},  
      "avg_review_score": {"$avg": "$avg_review"}  
    }  
  },  
  {  
    "$sort": {  
      "avg_weight": -1  
    }  
  }  
]  
  
opt_explain = db_opt.command({
```

```

    "explain": {
      "aggregate": "products",
      "pipeline": pipeline_opt,
      "cursor": {}
    },
    "verbosity": "executionStats"
  })

```

Evidencias

```

===== PRODUCTS RAW =====

Plan utilizado      : PROJECTION_DEFAULT
Docs Examined      : 65902
Keys Examined       : 0
Execution Time (ms) : 183

===== PRODUCTS OPTIMIZADO =====

Plan utilizado      : PROJECTION_DEFAULT
Docs Examined      : 27356
Keys Examined       : 27356
Execution Time (ms) : 156

```

Order_reviews

Esquemas de documentos

```

{
  $jsonSchema: {
    bsonType: 'object',
    required: [
      '_id',
      'order_id',
      'review_score'
    ],
    properties: {
      _id: {
        bsonType: 'string'
      },
      order_id: {
        bsonType: 'string',
        description: 'Llave relacional al Write Model'
      },
      review_score: {
        bsonType: 'int',
        minimum: 1,
        maximum: 5,
        description: 'Obliga a que la calificación sea un entero del 1 al 5'
      },
      review_comment: {
        bsonType: 'object',
        description: 'Subobjeto opcional (Patrón Polimórfico)',
        properties: {

```

```

        title: {
            bsonType: 'string'
        },
        message: {
            bsonType: 'string'
        }
    },
    timestamps: {
        bsonType: 'object',
        properties: {
            creation_date: {
                bsonType: 'string'
            },
            answer_timestamp: {
                bsonType: 'string'
            }
        }
    }
}
}
}
}
}

```

Índices implementados

```

db_opt.order_reviews.create_index(
    [
        ("review_score", 1)
    ],
    name="idx_reviews_esr"
)

```

Se implementó un índice simple sobre review_score en la colección olist_optimized.order_reviews:

Campo indexado review_score (ascendente): El pipeline aplica un \$match de rango sobre este campo (review_score >= 4). Al indexarlo, MongoDB ejecuta un index range scan directo sobre los valores 4 y 5, accediendo únicamente a los documentos que cumplen la condición sin recorrer la colección completa.

Índice simple sobre campo de filtro principal: Aunque review_score tiene solo 5 valores posibles (1 a 5), el filtro >= 4 delimita el conjunto de trabajo de forma efectiva en una

colección cuyo volumen total ya fue reducido durante la transformación del esquema. En este contexto, un índice simple sobre el campo de filtro es suficiente para que el optimizador lo prefiera sobre un collection scan, como lo confirma `Keys Examined = 75,917` en los resultados. Adicionalmente, se utilizó la directiva `hint` en la llamada `explain` para forzar el uso del índice y medir su impacto real frente al RAW sin índice.

Alineación con el esquema optimizado: El campo `review_score` en `olist_optimized` está tipado como entero desde la carga (`fillna(1).astype(int)`), garantizando que las comparaciones numéricas del índice operen sobre tipos consistentes y evitando casteos implícitos que degradan el rendimiento en colecciones con tipos mixtos.

Aggregation pipelines

```
pipeline_raw = [
    {
        "$match": {
            "review_score": {"$gte": 4}
        }
    },
    {
        "$project": {
            "review_score": 1
        }
    },
    {
        "$group": {
            "_id": "$review_score",
            "total_reviews": {"$sum": 1}
        }
    },
    {
        "$sort": {
            "_id": -1
        }
    }
]

raw_explain = db_raw.command({
    "explain": {
        "aggregate": "order_reviews",
        "pipeline": pipeline_raw,
```

```
    "cursor": {}  
  },  
  "verbosity": "executionStats"  
})
```

Evidencias

```
===== ORDER_REVIEWS RAW =====  
  
Plan utilizado      : GROUP  
Docs Examined      : 198448  
Keys Examined      : 0  
Execution Time (ms) : 372  
  
===== ORDER_REVIEWS OPTIMIZADO =====  
  
Plan utilizado      : GROUP  
Docs Examined      : 75917  
Keys Examined      : 75917  
Execution Time (ms) : 113
```

Geolocation

Esquemas de documentos

```
{  
  $jsonSchema: {  
    bsonType: 'object',  
    required: [  
      '_id',  
      'geolocation_zip_code_prefix',  
      'location'  
    ],  
    properties: {  
      _id: {  
        bsonType: 'string'  
      },  
      geolocation_zip_code_prefix: {  
        bsonType: 'int'  
      },  
      location: {  
        bsonType: 'object',  
        required: [  
          'type',  
          'coordinates'  
        ],  
        description: 'Estructura GeoJSON estricta',  
        properties: {  
          type: {  
            'enum': [  
              'Point'  
            ],  
            description: 'Solo se permite el valor \'Point\''  
          }  
        }  
      }  
    }  
  }  
}
```

```

    },
    coordinates: {
      bsonType: 'array',
      minItems: 2,
      maxItems: 2,
      description: 'Debe ser un arreglo exacto de [Longitud, Latitud]',
      items: {
        bsonType: [
          'double',
          'int'
        ]
      }
    }
  },
  city: {
    bsonType: 'string'
  },
  state: {
    bsonType: 'string'
  }
}
}
}

```

Índices implementados

```

db_opt.geolocation.create_index(
  [
    ("state", 1),
    ("city", 1)
  ],
  name="idx_state_city"
)

```

Se implementó un índice compuesto sobre state y city en la colección

olist_optimized.geolocation:

Campo líder state (ascendente): El pipeline aplica un \$match de igualdad exacta sobre state = "SP". Al ser el campo líder del índice, MongoDB ejecuta un index range scan directo sobre ese valor, accediendo únicamente a los documentos del estado filtrado sin recorrer el resto de la colección.

Segundo campo city (ascendente): Al incluir city en el índice, la información necesaria para la etapa \$project posterior queda disponible directamente en la estructura del índice. Esto permite al motor leer el valor de ciudad sin necesidad de acceder al documento completo en disco, habilitando un acceso de tipo index-covered para esa etapa del pipeline.

Decisión sobre el campo líder: Aunque state tiene baja cardinalidad (27 estados en Brasil), un filtro de igualdad exacta sobre un valor específico es suficiente para que el índice sea selectivo sobre la colección deduplicada. La combinación con city como segundo campo hace que el índice soporte tanto el filtrado por estado como el agrupamiento posterior por ciudad, cubriendo las dos operaciones principales del pipeline en una única estructura de índice.

Aggregation pipelines

```
pipeline_opt = [
    {
        "$match": {
            "state": "SP"
        }
    },
    {
        "$project": {
            "city": 1
        }
    },
    {
        "$group": {
            "_id": "$city",
            "total_locations": {
                "$sum": 1
            }
        }
    },
    {
        "$sort": {
            "total_locations": -1
        }
    }
]

opt_explain = db_opt.command({
    "explain": {
        "aggregate": "geolocation",
```

```
    "pipeline": pipeline_opt,  
    "cursor": {}  
  },  
  "verbosity": "executionStats"  
})
```

Evidencias

```
===== GEOLOCATION RAW =====  
  
Plan utilizado      : GROUP  
Docs Examined      : 2000326  
Keys Examined      : 0  
Execution Time (ms) : 1783  
  
===== GEOLOCATION OPTIMIZADO =====  
  
Plan utilizado      : GROUP  
Docs Examined      : 6349  
Keys Examined      : 6349  
Execution Time (ms) : 11
```

Replica Sets

Dentro de la arquitectura de la base de datos se hace uso de Replica Sets para garantizar la continuidad operativa e integridad de datos permitiendo así que el sistema tolere fallos o interrupciones de red.

Se configura un grupo de 3 nodos para cada segmento de datos así:

- **Nodo primario:** Nodo encargado de procesar operaciones de lectura y escritura manteniendo la consistencia.
- **Nodos secundarios:** Instancias con copias de los datos del nodo primario sincronizadas bajo Operations Log.
- **Estrategia Failover:** En caso de pérdida del nodo primario, el conjunto replica selecciona automáticamente uno de los dos nodos secundarios como primario.

Sharding

Para gestionar el crecimiento de la base de datos, se diseñó una arquitectura de tres pilares:

- **Shards:** Cada shard se configura como un Replica Set independiente asegurando una alta disponibilidad.

- Config Servers: Se configura como un Replica Set de 3 nodos que almacenan metadatos y el mapa de distribución de datos asegurando la disponibilidad de estos.
- Mongos: Estos reciben peticiones de los client, consultan a los Config Servers mapeando la ubicación de los datos y enrutan al Shard respectivo para el usuario final

Este diseño permite una correcta escalabilidad permitiendo al cluster crecer linealmente añadiendo nuevos shards conforme la cantidad de datos aumenta. Así mismo, el balanceador interno de MongoDB redistribuye los chunks automáticamente garantizando una carga de trabajo óptima.

Evidencias cuantitativas de mejoras de rendimiento

PostgreSQL: Tablas comparativas y gráficas de mejora

Los siguientes cuadros consolidan las métricas reales y el comportamiento del planificador de PostgreSQL antes y después de aplicar las optimizaciones descritas

SUPABASE					
Consulta	Antes		Después		% Eficiencia
	Plan de ejecución	Tiempo	Plan de ejecución	Tiempo	
Búsqueda de categorías	Seq Scan	0.04 ms	Seq Scan	0.04 ms	Sin cambio
Búsqueda de ciudades	Seq Scan	3.11 ms	Bitmap Heap Scan y Bitmap Or	1.49 ms	52%
Historial de compras de un cliente	Seq Scan y hash join	29.32 ms	Index Scan	8.05 ms	72.5%
Reporte de ventas totales	Seq Scan, hash join y sort	223.32 ms	Seq Scan, hash join y sort	228.29 ms	Sin cambio
Reporte de ventas canceladas	Seq Scan, hash join y sort	15.71 ms	Index Scan, Seq Scan, hash join y sort	5.77 ms	63%

COLAB					
Consulta	Antes		Después		% Eficiencia
	Plan de ejecución	Tiempo	Plan de ejecución	Tiempo	
Búsqueda de categorías	Seq Scan	1.262 ms	Seq Scan	0.056 ms	97%
Búsqueda de ciudades	Seq Scan	108.203 ms	Bitmap Heap Scan y Bitmap Or	2.294 ms	98%
Historial de compras de un cliente	Seq Scan y hash join	1277.599 ms	Index Scan	1014.020 ms	21%

Reporte de ventas totales	Seq Scan, hash join y sort	297.056 ms	Seq Scan, hash join y sort	239.099 ms	19%
Reporte de ventas canceladas	Seq Scan, hash join y sort	33.008 ms	Index Scan, Seq Scan, hash join y sort	23.016 ms	30%



MongoDB: Métricas de executionTimeMillis y efficiency ratios

Products

Métrica	RAW	Optimizado	Variación
Plan de ejecución	PROJECTION_DEF AULT	PROJECTION_DEF AULT	Sin cambio de plan
Docs Examined	65,902	27,356	↓ 58.5%
Keys Examined	0	27,356	Índice utilizado
executionTimeMillis	183 ms	156 ms	↓ 14.8%
Index Efficiency Ratio		1.00 (27,356 / 27,356)	Sin falsos positivos

Order Reviews

Métrica	RAW	Optimizado	Variación
Plan de ejecución	GROUP	GROUP	Sin cambio de plan
Docs Examined	198,448	75,917	↓ 61.7%
Keys Examined	0	75,917	Índice utilizado
executionTimeMillis	372 ms	113 ms	↓ 69.6%
Index Efficiency Ratio		1.00 (75,917 / 75,917)	Sin falsos positivos

Geolocation

Métrica	RAW	Optimizado	Variación
Plan de ejecución	GROUP	GROUP	Sin cambio de plan
Docs Examined	2,000,326	6,349	↓ 99.7%
Keys Examined	0	6,349	Índice utilizado
executionTimeMillis	1,783 ms	11 ms	↓ 99.4%
Index Efficiency Ratio		1.00 (6,349 / 6,349)	Sin falsos positivos

Interpretación de resultados y análisis de impacto

Products

La reducción del 58.5% en documentos examinados es consecuencia directa del índice compuesto `idx_products_partial`: el `$match` sobre `physical_specs.weight_g > 200` utiliza el campo líder del índice para descartar los productos de menor peso mediante un `index range scan`, sin recorrer la colección completa. El `Keys Examined` = 27,356 igual a `Docs Examined` confirma que el índice operó sin falsos positivos.

La mejora en tiempo de 183 ms a 156 ms (14.8%) es más moderada que en las otras colecciones porque el cuello de botella principal está en las etapas `$addFields` y `$group`, que

procesan datos en memoria sobre el conjunto ya filtrado y no se benefician directamente del índice. La ganancia real del índice se expresa principalmente en la reducción de I/O, visible en Docs Examined.

Order Reviews

La reducción del 61.7% en documentos examinados resulta de la acción combinada del índice `idx_reviews_esr` y el menor volumen de la colección optimizada. La colección `olist_raw.order_reviews` contiene 198,448 documentos producto de duplicados y registros sin depurar, mientras que `olist_optimized` redujo ese volumen durante la carga mediante limpieza de nulos y aplicación del patrón polimórfico para documentos sin comentarios. Sobre ese conjunto reducido, el índice permite un index scan directo en lugar de un collection scan completo.

La mejora en tiempo es la más significativa de las colecciones con esquema similar: de 372 ms a 113 ms (69.6%), una combinación del menor volumen base de la colección optimizada y del acceso indexado activado por el hint aplicado en el pipeline. El Keys Examined = 75,917 igual a Docs Examined confirma que el índice no generó lecturas desperdiciadas.

Geolocation

Geolocation concentra la mejora más significativa de las tres colecciones. La colección `olist_raw.geolocation` acumula 2,000,326 documentos debido a que el CSV original incluye múltiples registros por prefijo de código postal con variaciones menores de coordenadas. La colección optimizada resuelve esto en el proceso de carga: al agrupar por `zip_code_prefix` y conservar únicamente la primera ocurrencia, se generan 6,349 documentos únicos, uno por prefijo postal.

Sobre esta base deduplicada, el índice compuesto `idx_state_city` permite que el \$match sobre `state = "SP"` opere mediante un index scan directo, confirmado por Keys Examined = Docs Examined = 6,349. La relación 1:1 entre claves e índices examinados representa el escenario

de uso de índice más eficiente posible en MongoDB: cada entrada del índice leída corresponde exactamente a un documento válido sin ninguna lectura descartada.

El resultado es una reducción de 1,783 ms a 11 ms, equivalente a una mejora del 99.4% y un factor de aceleración de 162 veces frente a la versión sin optimizar. Esta mejora es producto de tres factores que operan de forma simultánea: rediseño del esquema con deduplicación, `$match` al inicio del pipeline que activa el índice desde la primera etapa, y cobertura del campo `city` en el índice que elimina accesos a disco adicionales durante `$project`.

Sincronización entre sistemas

Flujos de Sincronización

Para que la base de datos de MongoDB no quede aislada del estado operativo real de la plataforma, se define un modelo de Sincronización Circular Asíncrona Orientada a Eventos (Event-Driven). Este enfoque asegura que los motores cooperen sin bloquearse mutuamente:

Sincronización Transaccional a Consulta (Ida: PostgreSQL → MongoDB):

Cada vez que se ejecuta un cambio de estado importante en el Write Model (eje. una actualización administrativa de categorías), PostgreSQL confirma el cambio de manera ACID. Inmediatamente después del commit, el servicio correspondiente publica un evento en el Message Broker (Kafka). El microservicio del catálogo consume este mensaje en el trasfondo y realiza un update sobre el campo correspondiente en el documento NoSQL de MongoDB, actualizando el catálogo de forma transparente.

Sincronización de Interacción a Analítica (Vuelta: MongoDB → PostgreSQL):

Cuando un comprador genera datos que no alteran el flujo contable pero sí afectan al negocio como la inserción de una calificación de 5 estrellas en order_reviews, MongoDB persiste el documento de inmediato para asegurar baja latencia. En paralelo, un proceso de captura emite un evento del tipo ReviewSubmitted hacia Kafka. El motor consumidor de PostgreSQL toma este payload de forma asíncrona y actualiza la reputación acumulada del vendedor en la tabla relacional, protegiendo a la base de datos transaccional de la concurrencia masiva del front-end.

Estrategia de consistencia

Durante la sincronización entre sistemas se adopta un modelo de Consistencia Eventual utilizando un patrón Outbox Transaccional respondiendo así a la necesidad de mantener ambos mundos en sintonía.

- Entrega con Outbox: Para evitar eventos perdidos cuando algo falla, guardamos el cambio también en una tabla “Outbox” dentro de la misma transacción.
- Impotencia: Sin importar si se procesa el mismo mensaje una o diez veces, el resultado en base de datos será el mismo, eliminando la posibilidad de datos duplicados.
- Manejo de errores: Si un evento falla luego de una cantidad establecida de reintentos, lo movemos a una cola de errores (DLQ) permitiendo seguir sin interrupciones y resolver el problema manualmente.

Lecciones aprendidas

1. La línea base sin optimización es un punto de partida indispensable

Trabajar inicialmente con una base de datos sin índices adicionales, apoyándose únicamente en las restricciones de llaves primarias y foráneas, permitió identificar con precisión los cuellos de botella reales del sistema. Sin este punto de referencia, habría sido imposible cuantificar el impacto de cada optimización aplicada y justificar técnicamente las decisiones tomadas.

2. El tipo de índice debe responder al tipo de consulta

No existe un índice universalmente óptimo. La implementación demostró que los índices B-tree son adecuados para búsquedas por igualdad, rangos y ordenamiento sobre llaves foráneas y campos operativos, mientras que los índices GIN con la extensión `pg_trgm` son necesarios para búsquedas difusas con patrones LIKE. Seleccionar el tipo incorrecto puede no generar ninguna mejora o incluso degradar el rendimiento.

3. El optimizador de consultas toma decisiones basadas en costos, no en suposiciones

Una de las observaciones más valiosas del proyecto fue constatar que PostgreSQL no siempre utiliza un índice disponible. Cuando la consulta recupera una proporción significativa de la tabla, como ocurre con el estado delivered, el motor opta por un escaneo secuencial porque resulta más eficiente. Esta comprensión evita el error común de crear índices indiscriminadamente y obliga a medir antes de asumir.

4. El rediseño del esquema puede superar el impacto de cualquier índice

En MongoDB, la colección de geolocalización pasó de 2.000.326 documentos a 6.349 tras un proceso de deduplicación por prefijo postal. Esta decisión de diseño, combinada con un índice compuesto, produjo una reducción del 99,4% en el tiempo de ejecución. La lección es clara: un índice sobre un esquema mal diseñado tiene un

techo de mejora limitado, mientras que un esquema bien estructurado multiplica el efecto de cualquier optimización posterior.

5. Los índices compuestos deben construirse siguiendo el orden de las operaciones del pipeline

En MongoDB, la efectividad de un índice compuesto depende directamente de colocar como campo líder aquel sobre el que se aplica el filtro más selectivo. En la colección de productos, anteponer `physical_specs.weight_g` al índice permitió descartar documentos desde el primer stage del pipeline, convirtiendo la estructura en un covering index que evitó accesos adicionales a disco en etapas posteriores.

6. La consistencia eventual requiere diseño explícito, no es un comportamiento automático

La integración entre PostgreSQL y MongoDB dejó en evidencia que combinar un motor relacional con uno documental exige una estrategia de sincronización deliberada. La adopción del patrón Outbox Transaccional, junto con el uso de Kafka como broker de eventos y una cola de errores (DLQ) para reintentos fallidos, fue fundamental para garantizar que ningún evento se perdiera sin comprometer el rendimiento transaccional de ninguno de los dos sistemas.

7. El particionamiento debe justificarse con datos, no anticiparse por precaución

Durante el diseño se evaluó la posibilidad de particionar tablas como `orders` y `order_items`, pero el análisis reveló que el volumen actual de datos era gestionable con los índices existentes y que el particionamiento habría incrementado la complejidad operativa sin beneficios medibles. Esta decisión refuerza el principio de no optimizar prematuramente y mantener la simplicidad mientras el sistema no lo exija.

8. La arquitectura de alta disponibilidad debe planificarse desde el inicio

La configuración de Replica Sets con tres nodos por segmento de datos y el diseño de sharding con Config Servers y routers Mongos no son características que puedan añadirse fácilmente en etapas tardías del proyecto. Haberlas contemplado desde la fase de diseño permitió que la arquitectura soporte crecimiento lineal y tolere fallos sin interrupciones, sentando las bases para la escalabilidad futura de Ecommify.